

19.Naïve Bayes Classification for Bank Loan Prediction

Aim

To implement **Naïve Bayes classification** in Python for predicting whether a bank loan application will be **Approved or Rejected** based on customer details.

Algorithm

1. Import the required libraries.
2. Create a dataset containing customer loan details.
3. Separate the input features and target class.
4. Split the dataset into training and testing data.
5. Create a **Gaussian Naïve Bayes** classifier.
6. Train the model using the training data.
7. Predict the loan status for the test data.
8. Calculate the accuracy of the model.
9. Predict the loan status for a new customer.

Program:

```
from sklearn.naive_bayes import GaussianNB

from sklearn.model_selection import train_test_split

from sklearn.metrics import accuracy_score
```

```
X = [
    [25, 30000, 2, 1],
    [30, 40000, 3, 1],
    [35, 50000, 5, 2],
    [40, 60000, 7, 2],
    [45, 70000, 10, 3],
    [50, 80000, 12, 3],
    [28, 35000, 2, 1],
    [38, 55000, 6, 2],
```

```
[48, 75000, 11, 3],  
[55, 90000, 15, 3]  
]
```

```
y = [  
    "Rejected",  
    "Rejected",  
    "Approved",  
    "Approved",  
    "Approved",  
    "Approved",  
    "Rejected",  
    "Approved",  
    "Approved",  
    "Approved"  
]
```

```
X_train, X_test, y_train, y_test = train_test_split(  
    X, y, test_size=0.2, random_state=42  
)
```

```
model = GaussianNB()
```

```
model.fit(X_train, y_train)
```

```
y_pred = model.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)

print("Actual Loan Status:", y_test)
print("Predicted Loan Status:", y_pred)
print("Accuracy:", accuracy * 100, "%")
```

```
new_customer = [[32, 45000, 4, 2]]
```

```
prediction = model.predict(new_customer)

print("New Customer Details:", new_customer)
print("Predicted Loan Status:", prediction[0])
```

Output:

```
from sklearn.naive_bayes import GaussianNB
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
```

```
X = [
    [25, 30000, 2, 1],
    [30, 40000, 3, 1],
    [35, 50000, 5, 2],
```

```
[40, 60000, 7, 2],  
[45, 70000, 10, 3],  
[50, 80000, 12, 3],  
[28, 35000, 2, 1],  
[38, 55000, 6, 2],  
[48, 75000, 11, 3],  
[55, 90000, 15, 3]  
]
```

```
y = [  
    "Rejected",  
    "Rejected",  
    "Approved",  
    "Approved",  
    "Approved",  
    "Approved",  
    "Rejected",  
    "Approved",  
    "Approved",  
    "Approved"  
]
```

```
X_train, X_test, y_train, y_test = train_test_split(  
    X, y, test_size=0.2, random_state=42
```

)

model = GaussianNB()

model.fit(X_train, y_train)

y_pred = model.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)

print("Actual Loan Status:", y_test)

print("Predicted Loan Status:", y_pred)

print("Accuracy:", accuracy * 100, "%")

new_customer = [[32, 45000, 4, 2]]

prediction = model.predict(new_customer)

print("New Customer Details:", new_customer)

print("Predicted Loan Status:", prediction[0])

Output:

```
y_pred = model.predict(x_test)

accuracy = accuracy_score(y_test, y_pred)

print("Actual Loan Status:", y_test)
print("Predicted Loan Status:", y_pred)
print("Accuracy:", accuracy * 100, "%")

new_customer = [[32, 45000, 4, 2]]

prediction = model.predict(new_customer)

print("New Customer Details:", new_customer)
print("Predicted Loan Status:", prediction[0])
```

▼ ... Actual Loan Status: ['Approved', 'Rejected']
Predicted Loan Status: ['Approved' 'Rejected']
Accuracy: 100.0 %
New Customer Details: [[32, 45000, 4, 2]]
Predicted Loan Status: Approved

Result Statement

Thus, **Naïve Bayes classification for Bank Loan Prediction** was successfully implemented in Python, and the loan application status of a new customer was successfully predicted.